

Funções Lambda

Módulo

$$|-1| = 1$$

$$\rightarrow \text{abs}(-1) = 1$$

Coordenadas Cartesianas $\rightarrow$ Coordenadas Polares

$$(x, y) \rightarrow (r, \theta)$$

$$r = \sqrt{x^2 + y^2}$$

$$\begin{aligned} (\sqrt{x}) &\rightarrow \text{math.sqrt}(x) \\ x^y &\rightarrow \text{pow}(x, y) \end{aligned}$$

$$\theta = \text{atan2}\left(\frac{y}{x}\right)$$

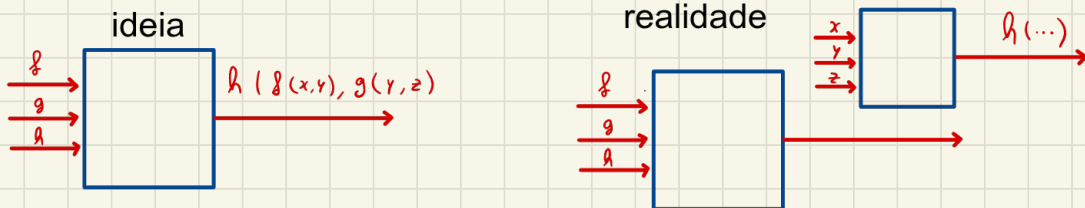
#Exercicio 4.4

```
cart2pol = lambda x, y: (math.sqrt(x**2 + y**2), math.atan2(y, x))
```

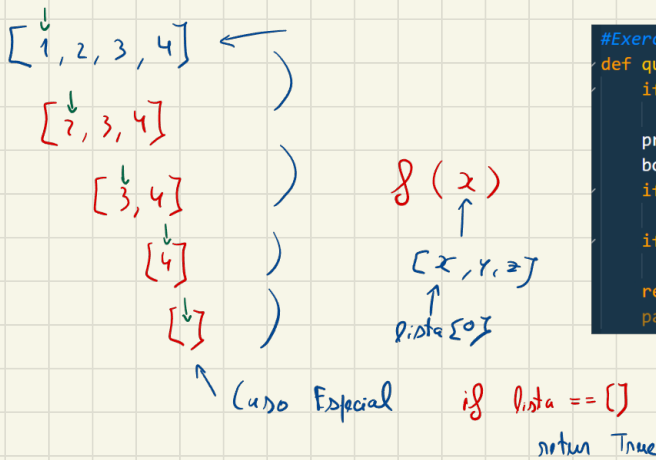
return



#Exercicio 4.5



```
ex5 = lambda f, g, h: lambda x, y, z: h(f(x, y), g(y, z))
```



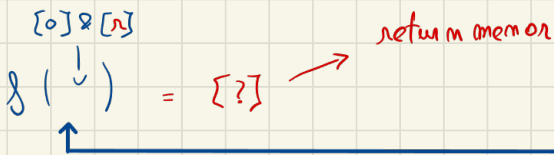
```

#Exercicio 4.6
def quantificador_universal(lista, f):
    if lista == []:
        return True
    primeiro_elemento = lista[0]
    booleano = quantificador_universal(lista[1:], f)
    if not f(primeiro_elemento):
        return False
    if not booleano:
        return False
    return True
    pass
  
```

Handwritten note: *função booleana*

Exercício 4.9

→ função binária



```

def menor_ordem(lista, f):
    if len(lista) == 1:
        return lista[0]
    resto = menor_ordem(lista[1:], f)
    # compara o primeiro com o menor do resto usando f
    if f(lista[0], resto):
        return lista[0]
    else:
        return resto
  
```

Handwritten note: *← Caso Base*

Exercício 4.10

Output: (1, [2, 3, 4]) Tuplo → os tuplos são imutáveis

```

def menor_e_resto_ordem(lista, f):
    if len(lista) == 1:
        return (lista[0], [])

    # divide a lista em primeiro e resto
    resultado = menor_e_resto_ordem(lista[1:], f)
    menor_do_resto = resultado[0]
    resto = resultado[1]

    # compara o primeiro com o menor encontrado no resto
    if f(lista[0], menor_do_resto):
        # lista[0] é o menor
        return (lista[0], [menor_do_resto] + resto)
    else:
        # menor_do_resto é o menor
        return (menor_do_resto, [lista[0]] + resto)
  
```

Exemplo [2, 4, 1, 7]:

- Chamada inicial: [2, 4, 1, 7] → divide em 2 e [4, 1, 7].
- Chamada recursiva em [4, 1, 7]: → divide em 4 e [1, 7].
- Chamada recursiva em [1, 7]: → divide em 1 e [7].
- Caso base: [7] → (7, []).
- Regressão recursão [1, 7]: compara 1 com 7 → menor é 1, resto = [7] → devolve (1, [7]).
- Regressão recursão [4, 1, 7]: compara 4 com 1 → menor é 1, resto = [4, 7] → devolve (1, [4, 7]).
- Regressão chamada inicial [2, 4, 1, 7]: compara 2 com 1 → menor é 1, resto = [2, 4, 7] → devolve (1, [2, 4, 7]).

Handwritten notes and diagrams show the recursive process and the final result: (1, [2, 3, 4]).

5.2)

lista = [1, 4, -1, 8]

Caso Base $\rightarrow$ o tamanho da lista é 1

usamos $\hookrightarrow$ len()

```
def ordenar_selecao(lista, ordem):
```

```
    if len(lista) <= 1: } Caso Base (Se o tamanho == 1)
        return lista
```

```
    meio = len(lista) // 2 } meio da lista
```

```
    esquerda = ordenar_selecao(lista[:meio], ordem)
```

```
    direita = ordenar_selecao(lista[meio:], ordem)
```

```
    resultado = []
```

```
    i, j = 0, 0
```

```
    while i < len(esquerda) and j < len(direita):
```

```
        if ordem(esquerda[i], direita[j]): } Se a comparação e/
            resultado.append(esquerda[i]) } esquerda e direita
            i += 1 } for verdadeira. Adiciona
```

```
        else: } a "esquerda" ao resultado
            resultado.append(direita[j])
            j += 1
```

Caso contrário
adiciona
esquerda

```
    resultado.extend(esquerda[i:]) } Serve para juntar os restos
    resultado.extend(direita[j:]) } das listas (Se sobrar!)
```

```
    return resultado
```

pass

meio
 $\downarrow$
LST = [x, y, z, w, k]
[: meio] [meio :]